

PixelSafe Project Report (JavaScript-only)

PixelSafe — Zero-Quality-Loss Photo Sharing Platform

Project Report (JavaScript-only build — no TypeScript)

1. Problem Statement

Most consumer sharing platforms (WhatsApp, Instagram DM, Telegram default mode, email clients) silently **re-encode and compress** every photo before transmission to save bandwidth and storage. This means:

- Resolution is downscaled
- JPEG quality is reduced (often to ~50–75%)
- EXIF metadata (camera settings, location, timestamps) is frequently stripped
- Repeated sharing (forward → forward → forward) causes **generational quality loss**, compounding each time

Goal: Build a platform that transfers the *exact original file bytes* from sender to receiver — with **zero recompression, zero resizing, and zero metadata loss** — and be able to cryptographically prove it.

2. Core Idea

The fix isn't "better compression" — it's **no compression at all**. If the file is never decoded into pixels and re-encoded, there is nothing to degrade. The platform's job is purely to move bytes from point A to point B and generate a shareable link in between.

Trust mechanism: Generate a SHA-256 hash of the file client-side before upload, and again after download. If the hashes match, quality loss is not just "avoided" — it's mathematically provable to the user.

3. Tech Stack (Zero-Cost, Plain JavaScript — No TypeScript)

Layer	Choice	Why
Object storage	Cloudflare R2	10GB free, zero egress fees (critical — files get downloaded, not just stored)
Backend / API	Cloudflare Workers (plain .js)	Generous free tier (100k req/day), integrates natively with R2, no cold-

Database	Supabase (Postgres)	sleep like other free-tier hosts Relational queries for expiry/cleanup jobs, generous free tier, no vendor lock-in
Frontend hosting	Cloudflare Pages	Free, auto-deploys from GitHub, same ecosystem as Workers/R2
Frontend framework	React (.jsx) + Vite	Lightweight, fast build, easy drag-and-drop UI — scaffolded with the plain react template, not react-ts
Hashing	Web Crypto API (built into browser)	Client-side SHA-256, no backend cost, no library needed
Domain	Optional — use free *.pages.dev / *.workers.dev subdomains initially	Custom domain (~\$10/yr) only needed later

Total monthly cost: \$0, until storage/traffic exceeds free tier limits.

No TypeScript anywhere — every file in this project is .js / .jsx. When scaffolding tools ask, always pick the JavaScript option (see Phase 1 below for exact prompts).

4. System Workflow

4.1 Upload Flow

1. User selects/drops a photo into the web app.
2. Browser reads the file as raw bytes (File/Blob API) — **no canvas rendering, no resizing step ever touches the image.**
3. Browser computes SHA-256 hash of the raw file (Web Crypto API) — this is the “quality fingerprint.”
4. Frontend requests a **presigned upload URL** from the Cloudflare Worker backend.
5. Backend generates the presigned URL (scoped to a unique object key) and returns it — the file itself never passes through your backend server, it goes straight browser → R2.
6. Browser uploads the raw file directly to R2 using the presigned URL.
7. Backend writes a metadata row to Supabase: {id, r2_key, filename, hash, size, uploaded_at, expires_at, download_count}.
8. Backend returns a short shareable link (e.g. pixelsafe.pages.dev/f/AB12cd) to the user.

4.2 Share & Download Flow

1. Receiver opens the link.
2. Frontend calls the Worker backend with the link ID.
3. Backend checks Supabase: does this ID exist? Has it expired? Has it hit its download/view limit?
4. If valid, backend generates a presigned **download URL** from R2 and returns it (or streams the file directly).
5. Receiver’s browser downloads the **exact original bytes**.
6. (Optional trust feature) Frontend recomputes the SHA-256 hash of the downloaded file and displays “✓ Verified — 0% quality loss” if

- it matches the original hash stored at upload time.
- Backend increments `download_count`; if a cleanup rule is hit (e.g., “delete after first download”), a scheduled Worker job removes the object from R2 and the row from Supabase.

4.3 Cleanup / Storage Management (to stay within free tier)

- A scheduled Cloudflare Worker (cron trigger) runs daily.
 - Queries Supabase for rows where `expires_at < now()`.
 - Deletes the corresponding object from R2 and removes the row.
 - Default expiry: e.g. 7 days, or “delete after first download” as a user-selectable option.
-

5. Database Schema (Supabase / Postgres)

```
create table files (  
  id text primary key,           -- short unique link ID  
  r2_key text not null,          -- object key in R2 bucket  
  filename text not null,  
  file_hash text not null,       -- SHA-256 of original file  
  size_bytes bigint not null,  
  uploaded_at timestamptz default now(),  
  expires_at timestamptz not null,  
  download_count int default 0,  
  delete_after_download boolean default false  
);
```

6. Step-by-Step Build Plan (JavaScript Only)

Phase 1 — Setup (Day 1)

- ☐ Create Cloudflare account → set up an R2 bucket
- ☐ Create Supabase project → run the schema above
- ☐ Scaffold frontend:

```
npm create vite@latest pixelsafe -- --template react
```

(select **react**, *not* react-ts, when prompted)

- ☐ Scaffold backend:

```
npm create cloudflare@latest
```

When asked “Do you want to use TypeScript?” → answer **No**. This generates `src/index.js` instead of `index.ts`.

Phase 2 — Upload Pipeline (Day 2-3)

- ☐ Build drag-and-drop / file-picker UI in plain JSX (no prop types via TS interfaces — use simple runtime checks or PropTypes if you want light validation)
- ☐ Implement client-side SHA-256 hashing using the native Web Crypto API:

```
async function hashFile(file) {  
  const buffer = await file.arrayBuffer();  
  const hashBuffer = await crypto.subtle.digest('SHA-256',  
  buffer);
```

```

    return Array.from(new Uint8Array(hashBuffer))
      .map(b => b.toString(16).padStart(2, '0'))
      .join('');
  }

```

- ☐ Write Worker endpoint (src/index.js): POST /api/upload-url → generates presigned R2 upload URL + inserts Supabase row
- ☐ Wire frontend to upload directly to R2 using the presigned URL
- ☐ Generate and display the shareable short link

Phase 3 — Download Pipeline (Day 4)

- ☐ Build the /f/:id receiver page (plain JSX route)
- ☐ Worker endpoint: GET /api/file/:id → validates expiry/limits, returns presigned download URL
- ☐ Frontend downloads file, recomputes hash, shows verification badge
- ☐ Increment download count, trigger delete-after-download logic if enabled

Phase 4 — Cleanup & Limits (Day 5)

- ☐ Cloudflare Worker cron trigger for expired file cleanup (plain .js scheduled handler)
- ☐ Add per-file expiry selector in upload UI (e.g. 1 day / 7 days / delete after first view)
- ☐ Add basic abuse protection (rate limiting uploads per IP, max file size cap)

Phase 5 — Polish (Day 6-7)

- ☐ Add upload/download progress bars
 - ☐ Support batch upload (zip client-side or list of links)
 - ☐ Add “Verified — 0% quality loss” hash-match UI badge
 - ☐ Deploy frontend to Cloudflare Pages, connect custom domain (optional)
 - ☐ Basic mobile responsiveness / installable PWA manifest
-

7. Why Skipping TypeScript Is Fine Here

- The whole backend is essentially **one database table** (files) — the type-safety benefit TypeScript usually provides (catching mismatched field names, wrong types passed between functions) matters far less at this scale.
 - Cloudflare Workers, Vite, React, and Supabase’s JS client all work identically well with plain JavaScript — none of them require TS to function.
 - You lose autocomplete-on-types and compile-time error catching, which just means being a bit more careful with field names (expires_at consistently, not sometimes expiresAt) and testing upload/download manually as you build each phase.
-

8. Differentiators (What Makes This Stand Out)

- **Provable losslessness** — most “no compression” claims are unverifiable; the SHA-256 hash match is concrete proof shown to the user.
- **Format-agnostic for free** — since nothing is decoded, RAW/HEIC/TIFF work automatically, unlike apps that only optimize

for JPEG/PNG.

- **Zero infrastructure cost** — sustainable to run indefinitely on free tiers with sane expiry defaults.
 - **Auto-expiring by design** — avoids the classic “abandoned file storage” problem that kills most free-tier file-sharing side projects.
-

9. Future Extensions (Optional, Later)

- Multi-file/album sharing with a single link
- Optional password-protected links
- QR code generation for cross-device sharing (phone → laptop)
- Browser extension for one-click “share original quality” from any webpage